



CENG 334

Introduction to Operating Systems

Spring 2025 - 2026

Homework 2 - Crossroad Synchronization

Due date: April 20, 2026, Monday, 23:59

1 Introduction

In real-world operating systems, managing concurrent access to shared resources without causing deadlocks, race conditions, or starvation is a fundamental challenge. A classic visualization of this problem is a traffic crossroad. When multiple independent entities (cars/threads) attempt to use a mutually exclusive shared space (the crossroad), the system must enforce strict synchronization rules to maintain safety and efficiency.

In this assignment, you will implement a thread-safe traffic controller using POSIX threads, mutexes, and condition variables. You will build a framework that evaluates changing queues, enforces lane and arrival order priorities, and allow parallel traffic without violating safety constraints. This will require a deep understanding of the Monitor synchronization pattern and careful management of thread blocking and signaling.

2 Crossroad Rules

You are tasked with simulating a traffic crossroad consisting of H horizontal lanes and V vertical lanes. Each car in the simulation is represented by a separate thread. Lanes with the same direction (ie. 2 horizontal lanes) are called parallel lanes and lanes with different directions (ie. 1 horizontal and a vertical lane) are called crossing lanes. Cars that are currently passing through the crossroad are called driving, the cars that are waiting their turn to drive are called waiting, cars not yet at or around the crossroad are called away. Thus, an away car arrives at the crossroad and becomes waiting, then enters the crossroad and becomes driving, and finally exits the crossroad and becomes away again. Waiting cars will enter the crossroad according to strict traffic and priority rules, drive across the crossroad, and then exit before potentially returning.

To simplify the problem, the traffic flow is heavily restricted:

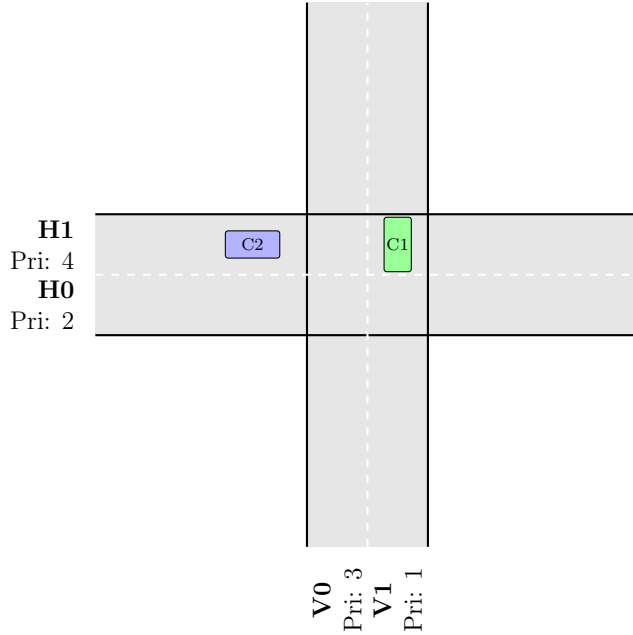
- **Simplified Traffic Flow:** Cars only travel in two orthogonal directions (e.g., exclusively West-to-East for horizontal lanes, and South-to-North for vertical lanes). There is no oncoming traffic from opposite directions, and cars cannot make turns at the crossroad.

Your program must manage the state of the crossroad to prevent collisions, enforce lane priorities, and ensure that cars do not starve or busy-wait.

The cars passing through the crossroad must obey the following rules.

- **No Collisions:** If the path of two cars within the crossroad intersects, a collision occurs. There are two type of possible collisions:
 - **Side Collisions:** While a car is driving, if another car from a crossing lane starts driving, a side collision happens (See Figure 1).

State 1: C1 from V1 is driving, C2 from H1 is waiting.



State 2: C2 from H1 enters before C1 leaves, a side collision occurs.

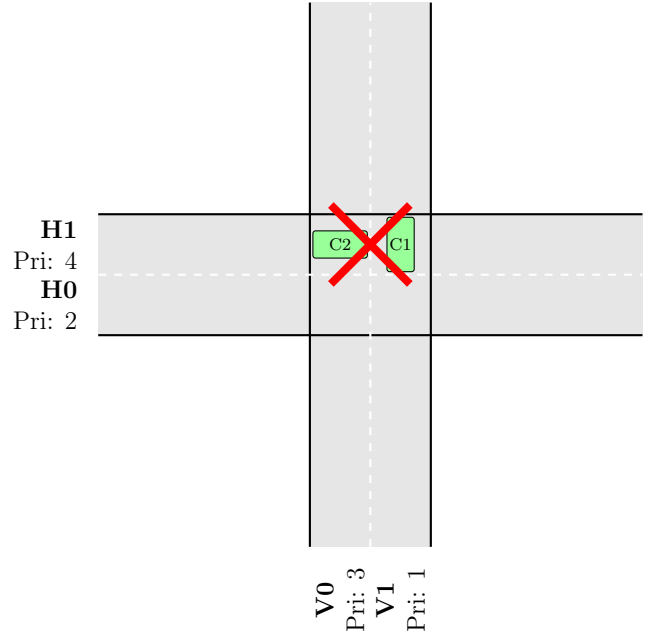
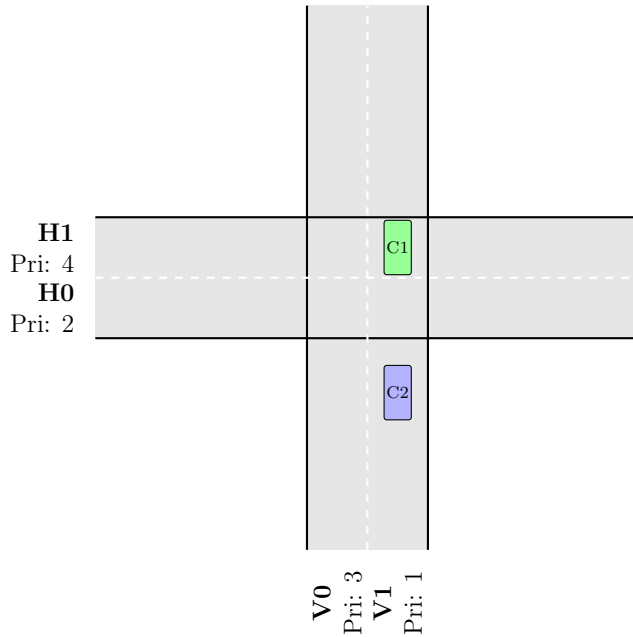


Figure 1: Side Collisions Example

- **No Rear Collision:** While a car is driving, if another car from the same lane starts driving, a rear collision happens (See Figure 2).
- **Parallel Traffic:** When a car is driving in a lane, other cars from parallel lanes can enter the crossroad (See Figure 3).
- **No FIFO Violation:** When a car in a lane enters the crossroad before a car waiting in the same lane that arrived before it, a FIFO Violation occurs (See Figure 4).
- **No Priority Violation:** Each lane has an assigned priority (A higher number means greater priority). When a car starts driving in a way that would block a car waiting in a crossing lane with a greater priority, a priority violation happens (See Figure 5).
 - **Equal Priority Tie-Breaking (Cross-Lane FIFO):** If cars are waiting in crossing lanes that share the exact same priority level, their right-of-way is resolved by their arrival time (strict FIFO).

For example, assume 2 horizontal lanes and a vertical lane both have a priority of 1. If cars arrive in the exact order of Car 1 (horizontal 1), Car 2 (vertical), and Car 3 (horizontal 2), they must enter the crossroad in that exact sequence: Car 1, then Car 2, then Car 3. Car 3 cannot enter in parallel with Car 1 because Car 2 arrived before Car 3 and therefore holds the right-of-way over it.

State 1: C1 from V1 is driving, C2 from V1 is waiting.



State 2: C2 from V1 enters before C1 leaves, a rear collision occurs.

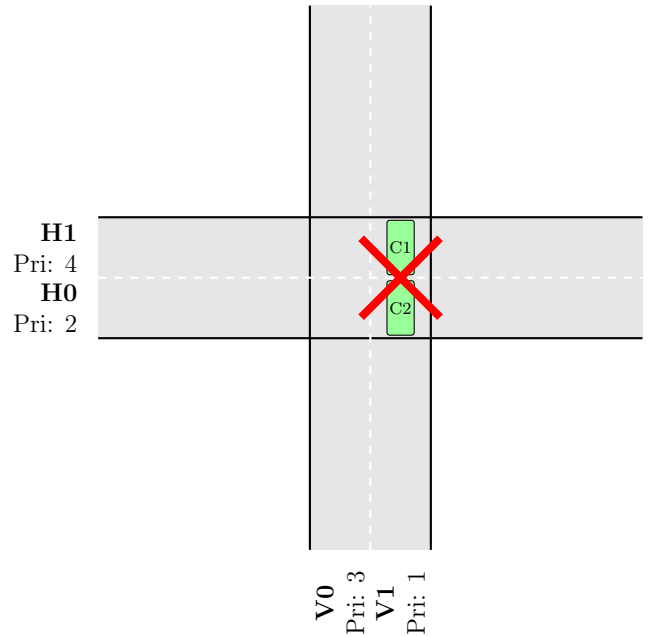


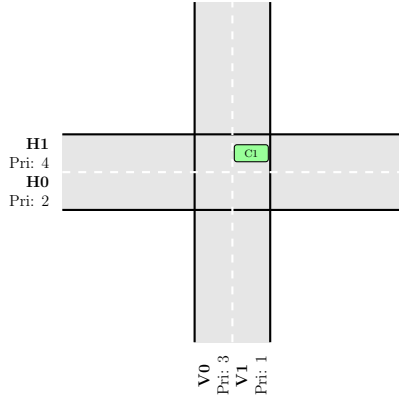
Figure 2: Rear Collisions Example

- **No Unnecessary Wait:** A car that is legally allowed to enter the crossroad must do so in a timely manner. A car is considered "legally allowed" if:
 1. It is at the front of its lane's queue.
 2. Its required path through the crossroad is currently clear (the car can enter without causing a collision).
 3. Entering would not cause a Priority Violation (i.e., no higher-priority car is waiting for a crossing lane).

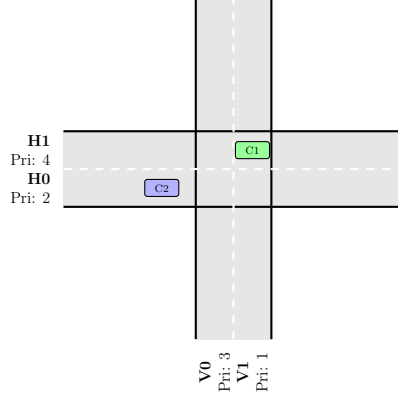
If the crossroad is available for a legally allowed car, and that car delays its entry beyond a strict, predefined grace period, an Unnecessary Wait violation occurs.

- **No Busy Waiting:** Threads must not consume CPU cycles while waiting for the crossroad to clear. You must use condition variables to block and wake threads efficiently.

State 1: C1 from H1 is driving.



State 2: C2 from H0 arrives.



State 3: C2 enters before C1 leaves, no collisions occur.

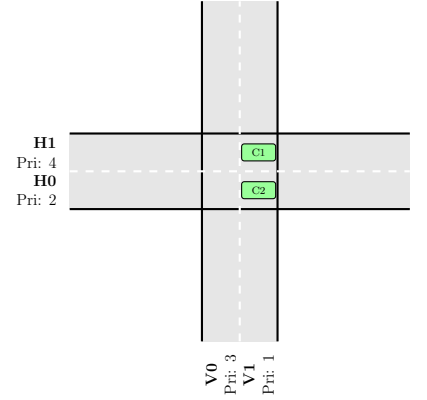
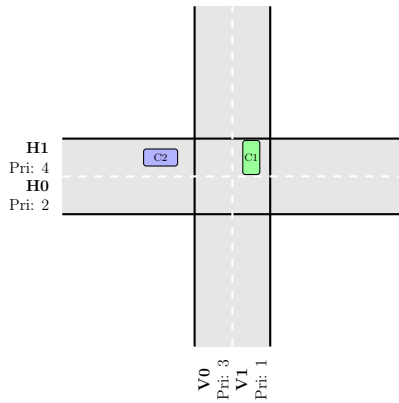
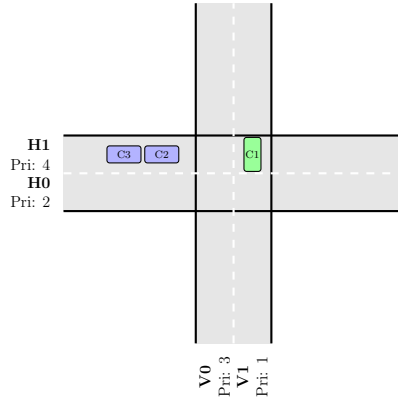


Figure 3: Parallel Traffic Example

State 1: C1 from V1 is driving, C2 from H1 is waiting.



State 2: C3 from H1 arrives, starts waiting as well.



State 3: C1 leaves, C3 enters before C2, a FIFO Violation occurs.

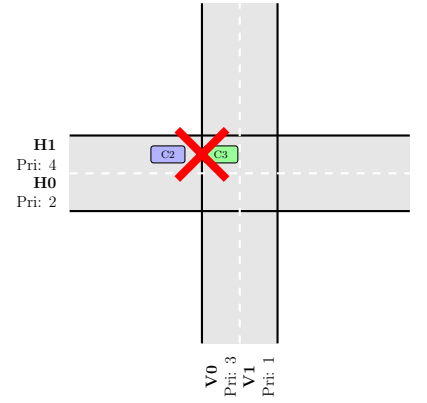
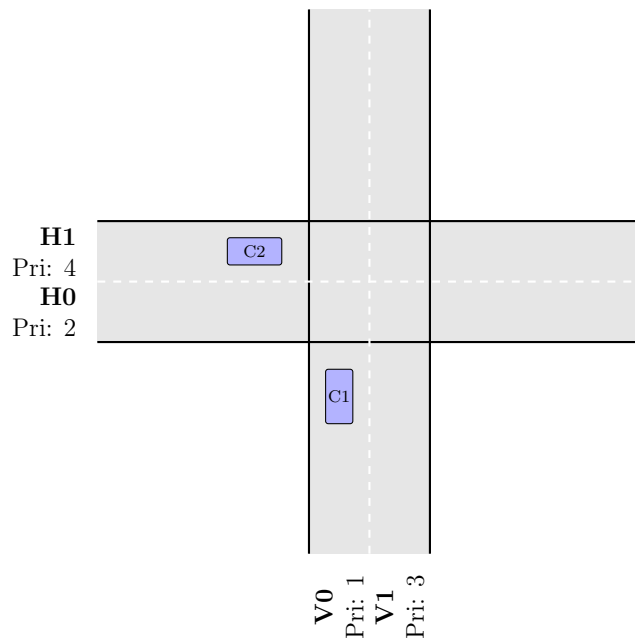


Figure 4: FIFO Violation Example

State 1: C1 (Pri 1) and C2 (Pri 4) are both waiting.



State 2: C1 enters before C2. C1 (Pri 1) blocks C2 (Pri 4), causing a Priority Violation.

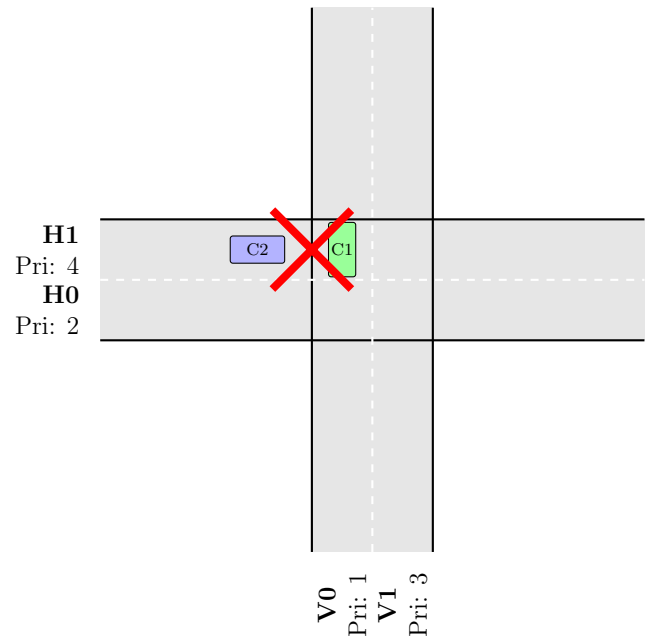
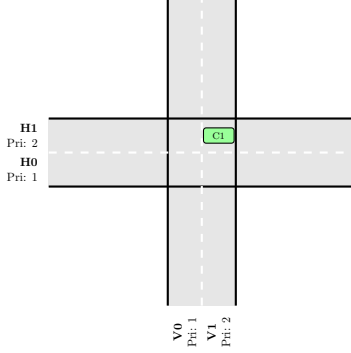


Figure 5: Priority Violation Example

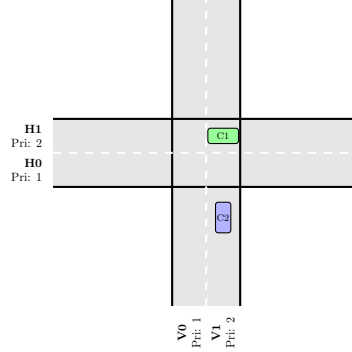
2.1 Comprehensive Execution

To fully illustrate how Priorities, Equal Priority Tie-Breaking (Cross-Lane FIFO), and Parallel Traffic rules interact, the following 9-state sequence traces a complex scenario where 5 cars arrive in rapid succession (See Figure 6.)

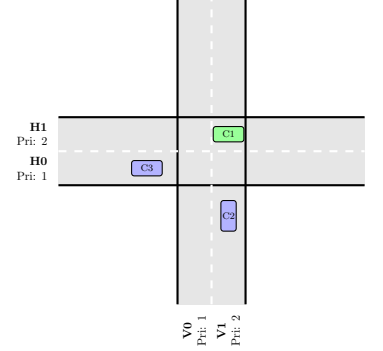
State 1: T=1
C1 arrives at H1 (Pri 2) and enters the empty crossroad.



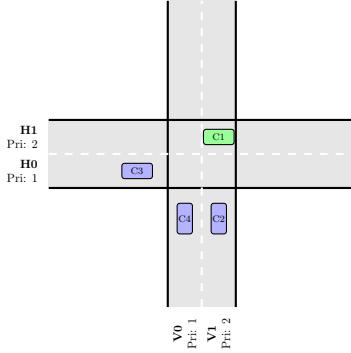
State 2: T=2
C2 arrives at V1 (Pri 2) and waits because C1 is currently crossing it.



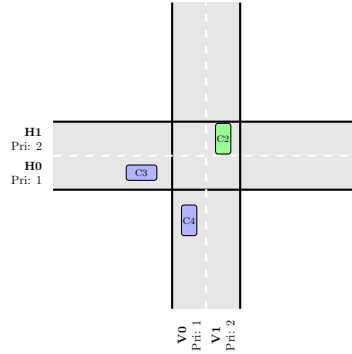
State 3: T=3
C3 arrives at H0 (Pri 1). It is parallel to C1, but **must wait** to prevent starving C2 (Pri 2).



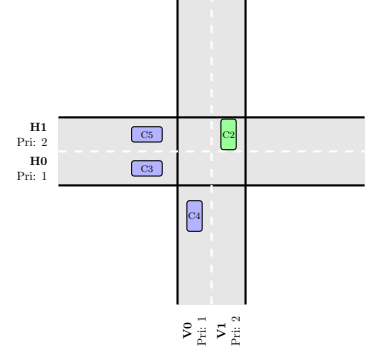
State 4: T=4
C4 arrives at V0 (Pri 1). It crosses H1 and H0, so it waits.



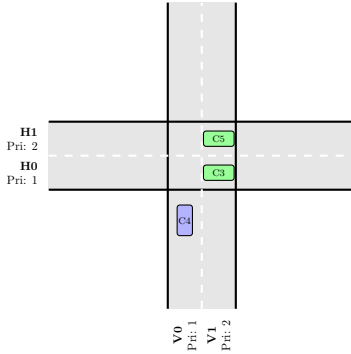
State 5: C1 Exits
C2 (Pri 2) enters. C4 is parallel to C2, but **must wait** because C3 (Pri 1) arrived before it (Cross-Lane FIFO).



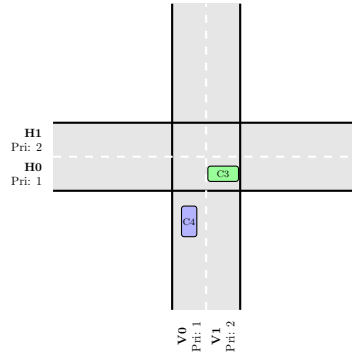
State 6: T=5
C5 arrives at H1 (Pri 2) and waits for C2.



State 7: C2 Exits
C5 (Pri 2) enters. C3 (Pri 1) enters parallel to C5, as it is first in its priority and no higher priority crossing cars are waiting.



State 8: C5 Exits
C3 is still driving. C4 must continue to wait as it crosses C3's path.



State 9: C3 Exits
The crossroad is clear, and C4 finally enters to complete the execution trace.

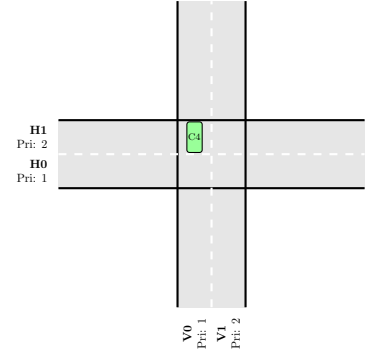


Figure 6: Full execution trace resolving Starvation, Priorities, and Cross-Lane FIFO.

3 Implementation Details

You are provided with a public testing environment and an output logging library (`hw2_output.h`). You are only allowed to implement your synchronization logic inside the provided `crossroad.cpp` (or `crossroad.c`) file.

3.1 The Output Library (`hw2_output.h`)

To ensure standardized grading and prevent logging race conditions, you must use the provided logging library. It defines the following enumerations:

- **Direction:** Represents the route of the car. Values are `DIR_HORIZONTAL` for horizontal travel and `DIR_VERTICAL` for vertical travel.
- **EventType:** Represents the car's action. Values are `ET_ARRIVE` for a car arriving at the crossroad, `ET_ENTER` for a car entering to the crossroad, and `ET_EXIT` for a car leaving the crossroad.

It also provides the logging function:

- `void hw2_write_output(int car_id, EventType event, Direction dir, int lane);`
This function safely logs the timestamped state of a car. To guarantee causal ordering in the grader, you **must** call this function from *inside* your critical sections (while holding your crossroad mutex).

3.2 The Student API (`crossroad.h`)

You must implement the following three functions:

- `void initialize_crossroad(int h_lanes, int v_lanes, int* h_pri, int* v_pri);`
Called once before any threads are spawned. You must dynamically allocate your state arrays, queues, and initialize your mutexes and condition variables here. `h_lanes` and `v_lanes` specify the number of horizontal and vertical lanes respectively, and `h_pri` and `v_pri` specify the priority of horizontal and vertical lanes in order respectively. `h_lanes` and `v_lanes` are guaranteed to be greater than 0 and less than or equal to 15, and `h_pri` and `v_pri` are guaranteed to have a number of elements equal to `h_lanes` and `v_lanes` respectively and have values greater or equal to 0 and less than or equal to 63. Both vertical and horizontal lanes may share the same priority.
- `void arrive_crossroad(int car_id, Direction dir, int lane);`
Called by a car thread when it arrives. This function must block until the car is legally allowed to enter according to the rules above. You **must** call `hw2_write_output` with `ET_ARRIVE` when the function begins, and `ET_ENTER` right before the function successfully returns.
- `void exit_crossroad(int car_id, Direction dir, int lane);`
Called by a car thread when it leaves the crossroad. This function must update the crossroad state, log the `ET_EXIT` event using `hw2_write_output`, and properly signal any waiting threads that are now eligible to enter based on the priority rules.

3.3 The Grading Environment

Your submission will be evaluated using a private grading environment. The following file replacement process will occur during grading:

- **Files Retained:** Only your `Makefile`, `crossroad.h`, `crossroad.c`, and/or `crossroad.cpp` will be kept. **Any other files in your submission folder will be deleted.**

- **Files Replaced:** We will inject our own private versions of `main.c/main.cpp`, `hw2_output.c`, and various `.txt` test case files. The `hw2_output.h` and any standard threading headers will remain identical to the ones provided to you.
- **The Consequence:** The private `main` function we use to test your code will spawn threads and simulate traffic in ways that may differ significantly from the provided public `main.cpp`. You must not rely on global variables defined in the public `main.cpp`, nor should you attempt to predict the thread life-cycle. Your logic must strictly rely on the parameters passed to your functions and the state of your synchronization primitives.

4 Grading

Your solution will be evaluated out of 100 points per testcase. Your overall assignment grade will be the average of your scores across all hidden testcases. For each testcase, the case will be graded 10 times and you will receive the median result. The grading is modular, meaning you can still earn partial credit even if you do not implement all the advanced priority rules perfectly. Your score for any individual testcase cannot drop below 0.

You start with 100 points. Deductions are applied for rule violations as follows:

- **Incomplete Execution (0 Points):** If your code deadlocks, crashes (e.g., segmentation fault), or fails to route all arriving cars to their exits, you will automatically receive 0 points for that testcase.
- **Collisions (-20 Points per occurrence):** Any side or rear collision will result in a 20-point deduction. This penalty stacks up to a maximum of 4 times (-80 points) per testcase.
- **Priority Violation (-20 Points):** If a lower-priority car illegally enters the crossroad while a higher-priority car is waiting at a crossing lane, you will receive a flat 30-point deduction. This is applied at most once per testcase.
- **Equal Priority Tie-Breaking Violation (-10 Points):** Failing to respect the cross-lane FIFO rule (ignoring the arrival time of crossing cars with the exact same priority) will result in a flat 10-point deduction. This is applied at most once per testcase.
- **Intra-Lane FIFO Violation (-20 Points):** If a car enters before another car that arrived earlier in the *exact same* lane, you will receive a flat 10-point deduction. This is applied at most once per testcase.

Liveness Penalty

If your code successfully routes all cars but the grader detects an **Unnecessary Wait** (leaving the crossroad artificially empty when eligible cars are waiting) or **Busy Waiting** (spinning CPU cycles without properly utilizing condition variables), your final calculated score for that testcase will be **halved**.

Example: If you have a perfectly synchronized crossroad with no collisions and perfect priority handling, but you used an inefficient `sleep()` or spinlock that triggered a Liveness Penalty, your score for that testcase will drop from 100 to 50.

5 Specifications

1. Your code must be written in C or C++.
2. Your implementation will be compiled and evaluated on the ineks, so you should make sure that your code works on them.

3. You may use `<pthread.h>`, `<semaphore.h>` and the `monitor.h` that we supplied to you for threads, mutexes, condition variables, and concurrency.
4. You can use standard C/C++ libraries that do not have concurrency related functionality.
5. Do not attempt to modify the thread lifecycle, `sleep()` timers, or the `main` function loop, as your code will be compiled against a private grader environment.
6. We have a zero-tolerance policy against cheating. All the code you submit must be your own work. Sharing code with your friends, using code from the internet or previous years' homework or using LLM generated code are all considered plagiarism and strictly forbidden.
7. Follow the course page on ODTUClass for possible updates and clarifications.
8. Please ask your questions on ODTUClass instead of sending an email for questions that do not contain code or solutions so that all may benefit.
9. **Late Submission:** Late submission is allowed but with a penalty of $5 \times \text{day} \times \text{day}$.

6 Tips and Tricks

- **Optional C++ Helper (`monitor.h`):** If you are writing your solution in C++, you are provided with `monitor.h`. This file contains a `Monitor` base class that wraps POSIX mutexes and condition variables using the RAII pattern. Using the `__synchronized__` macro can help you avoid accidental deadlocks caused by returning from a function without unlocking. Its use is entirely optional; you may use raw `pthread` calls if you prefer or if you are coding in pure C.
- Implementing a proper multi-threaded framework that ignores FIFO and Priority rules but properly handles collisions will enable you to get 50 points in cases they occur and full points from test cases that do not include them.

7 Submission

Submission will be done via ODTUClass. Create a gzipped tarball file named `hw2.tar.gz` that contains **only** your source code files (`crossroad.h`, `crossroad.c`, and/or `crossroad.cpp`) together with your `Makefile`. Your archive file should not contain any subfolders.

To enforce the black-box testing environment detailed in the Implementation section, your code will be extracted, stripped of unauthorized files, surrounded by a private testing environment, and evaluated using a sequence similar to the following:

```
# 1. Extract your submission
$ tar -xzf hw2.tar.gz

# 2. Delete everything except the allowed submission files
$ find . -type f ! -name 'Makefile' ! -name 'crossroad.h' \
  ! -name 'crossroad.c' ! -name 'crossroad.cpp' -delete

# 3. Inject the private grading files and test cases
$ cp /grader_env/main.cpp .
$ cp /grader_env/main.c .
$ cp /grader_env/hw2_output.c .
$ cp /grader_env/hw2_output.h .
```

```
$ cp /grader_env/monitor.h .  
$ cp /grader_env/testcases/*.txt .  
  
# 4. Compile and Run  
$ make all  
$ ./hw2
```

If there is a mistake in any of these steps (e.g., your **Makefile** is missing, your code fails to compile against the private **main.cpp/main.c**, or your archive extracts into a subfolder), you will lose 10 points.